`module` is already used by Node.js to refer to the current module, so something will have to change:

```
module Formulaic
  export Formulaic
```

CoffeeScript that can compile to module-supporting ECMAScript 6

Then elsewhere you `import...from`:

```
module QuoteApplication
  import { Formulaic } from Formulaic
```

CoffeeScript that can compile to module-supporting ECMAScript 6

Your CoffeeScript program has a compilation step, so regardless of the module system your CoffeeScript program uses today, the JavaScript that it generates can use a different module system tomorrow. Because the module system is one of the most likely areas that can still change, it's easier to move on to more defined areas such as `const` and `let`.

13.3.2. const and let

The variable-scoping rules catch out many programmers new to JavaScript. One of the reasons for this is that it has a syntax similar to C and Java but very different scoping rules. In order to provide some of the naming rules that people expect to find in JavaScript, two new keywords, `const` and `let`, are being added in ECMAScript 6. You'll notice that you almost use their semantics already.

const

Many CoffeeScript programs have a few variables that are written in uppercase:

```
ONE_SECOND = 1000

setInterval ->
  rocket.forward()
, ONE_SECOND
```

Why name this variable in uppercase? In JavaScript it's really just tradition handed down from programming languages with a C heritage. It's meant to